Summarize

Chapter 3. The Sidecar Pattern

The first single-node pattern is the sidecar pattern. The sidecar pattern is a single-node pattern made up of two containers. The first is the *application container*. It contains the core logic for the application. Without this container, the application would not exist. In addition to the application container, there is a *sidecar container*. The role of the sidecar is to augment and improve the application container, often without the application container's knowledge. In its simplest form, a sidecar container can be used to add functionality to a container that might otherwise be difficult to improve. Sidecar containers are coscheduled onto the same machine via an atomic *container group*, such as the `pod` API object in Kubernetes. In addition to being scheduled on the same machine, the application container and sidecar container share a number of resources, including parts of the filesystem, hostname, and network, and many other namespaces. A generic image of this sidecar pattern is shown in Figure 3-1.

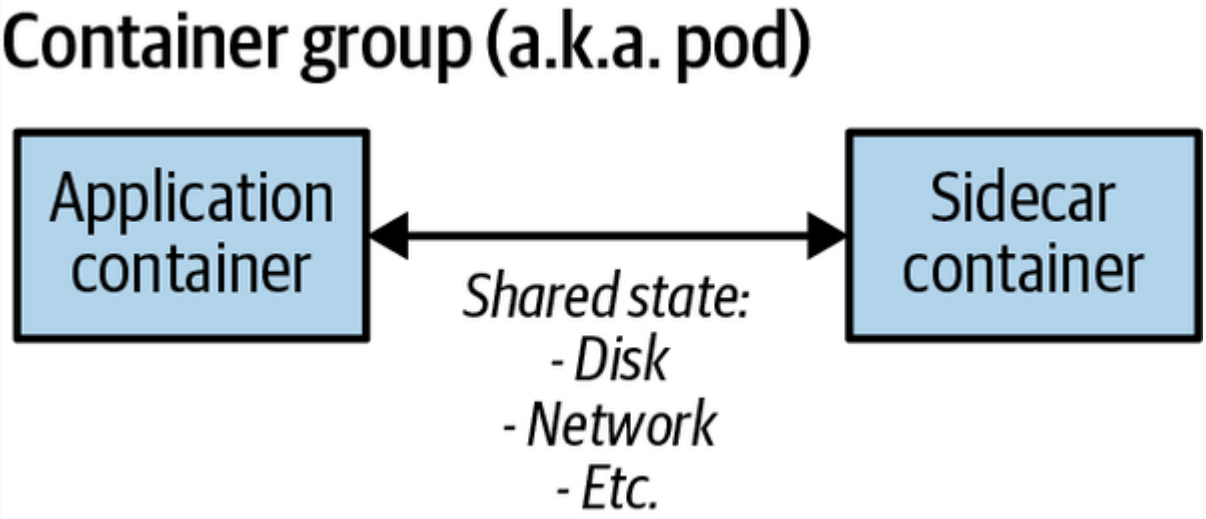


Figure 3-1. The generic sidecar pattern

An Example Sidecar: Adding HTTPS to a Legacy Service

Consider, for example, a legacy web service. Years ago, when it was built, internal network security was not as high a priority for the company, and thus, the application only services requests over unencrypted HTTP, not HTTPS. Due to recent security incidents, the company has mandated the use of HTTPS for all company websites. To compound the misery of the team sent to update this particular web service, the source code for this application was built with an old version of the company's build system, which no longer functions.

Containerizing this HTTP application is simple enough: the binary can run in a container with a version of an old Linux distribution on top of a more modern kernel being run by the team's container orchestrator. However, the task of adding HTTPS to this application is significantly more challenging. The team is trying to decide between resurrecting the old build system versus porting the application's source code to the new build system, when one of the team members suggests that they use the sidecar pattern to resolve the situation more easily.

The application of the sidecar pattern to this situation is straightforward. The legacy web service can be configured to serve exclusively on localhost (127.0.0.1), which means that only services that share the local network with the server will be able to access the service. Normally, this wouldn't be a practical choice because it would mean that no one could access the web service. However, using the sidecar pattern, in addition to the legacy container, we will add an `nginx` sidecar container. This `nginx` container lives in the same network namespace as the legacy web application, so it can access the service that is running on localhost.

At the same time, this `nginx` service can terminate HTTPS traffic on the external IP address of the pod and proxy that traffic to the legacy web application (see Figure 3-2). Since this unencrypted traffic is only sent via the local loopback adapter inside the container group, the network security team is satisfied that the data is safe. Likewise, by using the sidecar pattern, the team has modernized a legacy application without having to figure out how to rebuild a new application to serve HTTPS. A similar form of this pattern can also be used to add automatic certificate rotation, or even authentication and authorization to legacy web applications that may not be easy to modify.

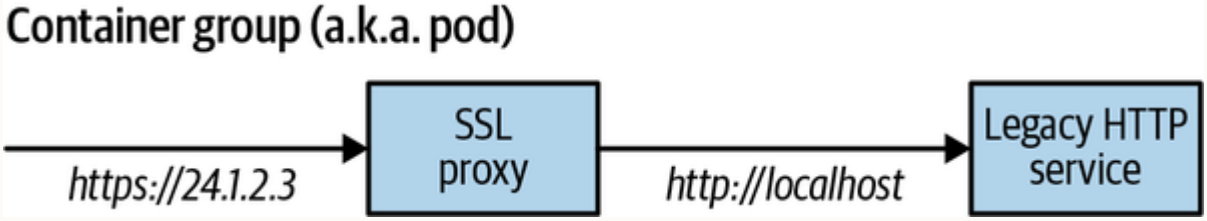


Figure 3-2. The HTTPS sidecar

Dynamic Configuration with Sidecars

Simply proxying traffic into an existing application is not the only use for a sidecar. Another common example is configuration synchronization. Many older applications use a configuration file for parameterizing the application; this may be a raw text file or something more structured like TOML, XML, JSON, or YAML. Many preexisting applications were written to assume that this file was present on the filesystem and read their configuration from there. However, in a cloud native environment it is often quite useful to use an API for updating configuration stored elsewhere. This allows you to do a dynamic push of configuration information via an API instead of manually logging in to every server and updating the configuration file using imperative commands. The desire for such an API is driven both by ease of use as well as the ability to add automation like rollback, which makes configuring (and reconfiguring) safer and easier. Kubernetes supports `ConfigMap` resources for exactly this purpose.

Similar to the case of HTTPS, new applications can be written with the expectation that configuration is a dynamic property that should be obtained using a cloud API, but adapting and updating an existing application can be significantly more challenging. Fortunately, the sidecar pattern again can be used to provide new functionality that augments a legacy application without changing the existing application. For the sidecar pattern shown in Figure 3-3, there again are two containers: the container that is the serving application and the container that is the configuration manager. The two containers are grouped together into a pod, where they share a directory between themselves. This shared directory is where the configuration file is maintained.

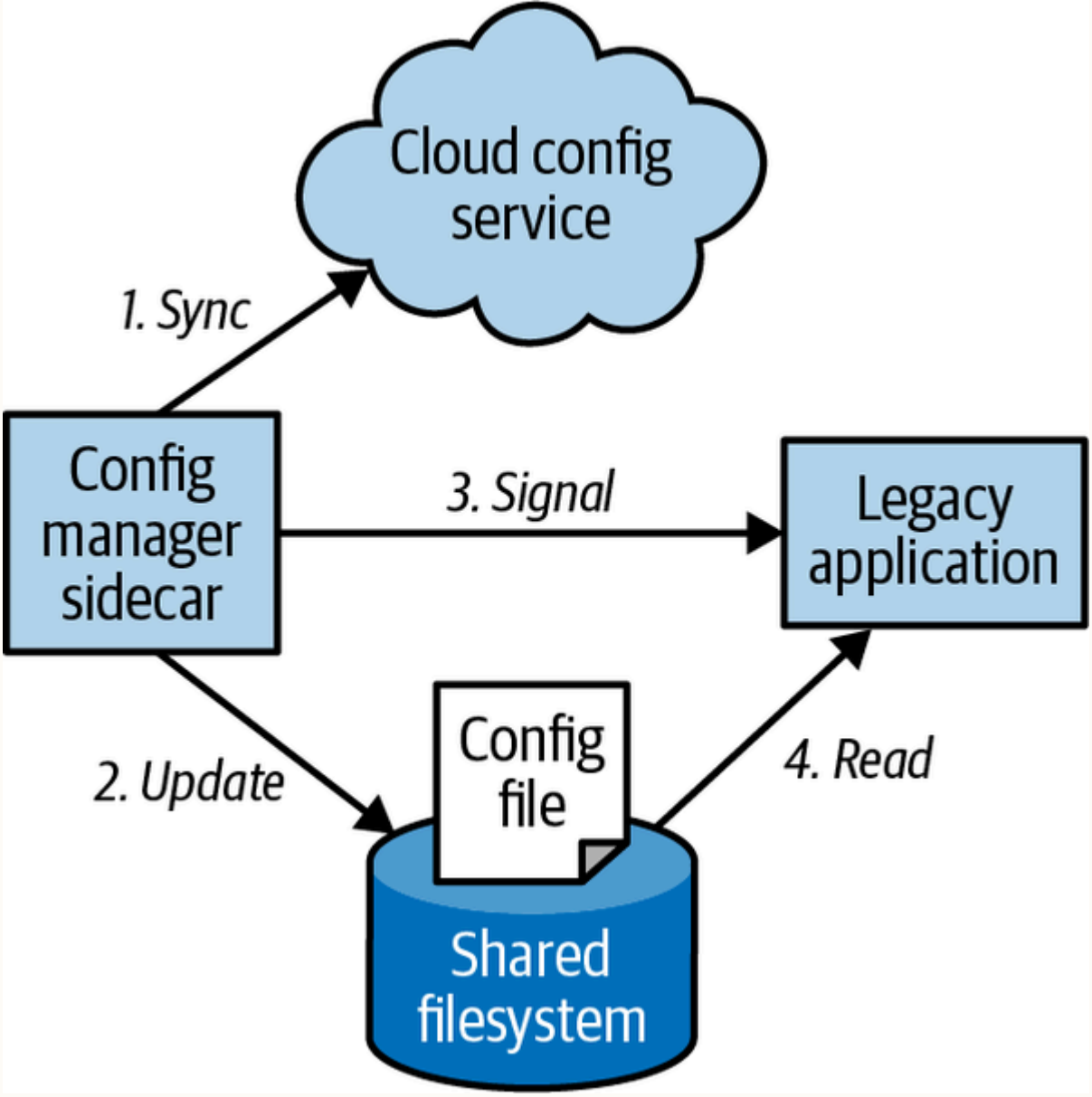


Figure 3-3. A sidecar example of managing a dynamic configuration

When the legacy application starts, it loads its configuration from the filesystem, as expected. This configuration is placed into the filesystem using a `ConfigMap` volume which places the current contents of a `ConfigMap` resource at a particular location in the filesystem. When the configuration manager starts, it examines the configuration API and looks for differences between the local filesystem and the configuration stored in the API. If there are differences, the configuration manager downloads the new configuration to the local filesystem and signals to the legacy application that it should reconfigure itself with this new configuration. The actual mechanism for this notification varies by application. Some applications actually watch the configuration file for changes, while others respond to a SIGHUP signal. In extreme cases, the configuration manager may send a SIGKILL signal to abort the legacy application. Once aborted, the container orchestration system will restart the legacy application, at which point it will load its new configuration. As with adding HTTPS to an existing application, this pattern illustrates how the sidecar pattern can help adapt pre-existing applications to more cloud native scenarios.

Modular Application Containers

At this point, you might be forgiven if you thought that the sole reason for the sidecar pattern to exist was to adapt legacy applications where you no longer wanted to make modifications to the original source code. While this is a common use case for the pattern, there are many other motivations for designing things using sidecars. One of the other main advantages of using the sidecar pattern is modularity and reuse of the components used as sidecars. In deploying any real-world, reliable application, there is functionality that you need for debugging or other management of the application, such as giving a readout of all of the different processes using resources in the container, similar to the `top` command-line tool.

One approach to providing this introspection is to require that each developer implement an HTTP `/topz` interface that provides a readout of resource usage. To make this easier, you might implement this webhook as a language-specific plug-in that the developer could simply link into their application. But even if done this way, the developer would be forced to choose to link it in, and your organization would be forced to implement the interface for every language it wants to support. Unless done with extreme rigor, this approach is bound to lead to variations among languages as well as a lack of support for the functionality when using new languages. Instead, this `topz` functionality can be deployed as a sidecar container that shares the process-ID (PID) namespace with the application container. This `topz` container can introspect all running processes and provide a consistent user interface. Moreover, you can use the orchestration system to automatically add this container to all applications deployed via the orchestration system to ensure that there is a consistent set of tools available for all applications running in your infrastructure.

Of course, as with any technical choice, there are trade-offs between this modular container-based pattern and rolling your own code into your application. The library-based approach is always going to be somewhat less tailored to the specifics of your application. This means that it may be less efficient in terms of performance, or that the API may require some adaptation to fit into your environment. I would compare these trade-offs to the difference between buying off-the-rack clothing versus bespoke fashion. The bespoke fashion will always fit you better, but it will take longer to arrive and cost more to acquire. As with clothes, for most of us it makes sense to buy the more general-purpose solution when it comes to coding. Of course, if your application demands extremes in terms of performance, you can always choose the handwritten solution.

Hands On: Deploying the topz Container

To see the `topz` sidecar in action, you first need to deploy some other container to act as the application container. Choose an existing application that you are running and deploy it using Docker:

```
$ docker run -d <my-app-image>
<container-hash-value>
```

After you run that image, you will receive the identifier for that specific container. It will look something like: `cccf82b85000...` If you don't have it, you can always look it up using the `docker ps` command, which will show all currently running containers. Assuming you have stashed that value in an environment variable named `APP_ID`, you can then run the `topz` container in the same PID namespace using:

```
$ docker run --pid=container:${APP_ID} \
   -p 8080:8080 \
   brendanburns/topz:db0fa58 \
   /server --addr=0.0.0.0:8080
```

This will launch the `topz` sidecar in the same PID namespace as the application container. Note that you may need to change the port that the sidecar uses for serving if your application container is also running on port `8080`. Once the sidecar is running, you can visit *http://localhost:8080/topz* to get a complete readout of the processes that are running in the application container and their resource usage.

You can mix this sidecar with any other existing container to easily get a view into how the container is using its resources via a web interface.

Building a Simple PaaS with Sidecars

The sidecar pattern can be used for more than adaptation and monitoring. It can also be used as a means to implement the complete logic for your application in a simplified, modular manner. As an example, imagine building a simple platform as a service (PaaS) built around the `git` workflow. Once you deploy this PaaS, simply pushing new code up to a Git repository results in that code being deployed to the running servers. We'll see how the sidecar pattern makes building this PaaS remarkably straightforward.

As previously stated, in the sidecar pattern there are two containers: the main application container and the sidecar. In our simple PaaS application, the main container is a Node.js server that implements a web server. The Node.js server is instrumented so that it automatically reloads the server when new files are updated. This is accomplished with the `nodemon` tool.

The sidecar container shares a filesystem with the main application container and runs a simple loop that synchronizes the filesystem with an existing Git repository:

```
#!/bin/bash

while true; do
  git pull
  sleep 10
done
```

Obviously, this script could be more complex, pulling from a specific branch instead of simply from HEAD. It is left purposefully simple to improve the readability of this example.

The Node.js application and Git synchronization sidecar are coscheduled and deployed together to implement our simple PaaS (Figure 3-4). Once deployed, every time new code is pushed to a Git repository, the code is automatically updated by the sidecar and reloaded by the server.

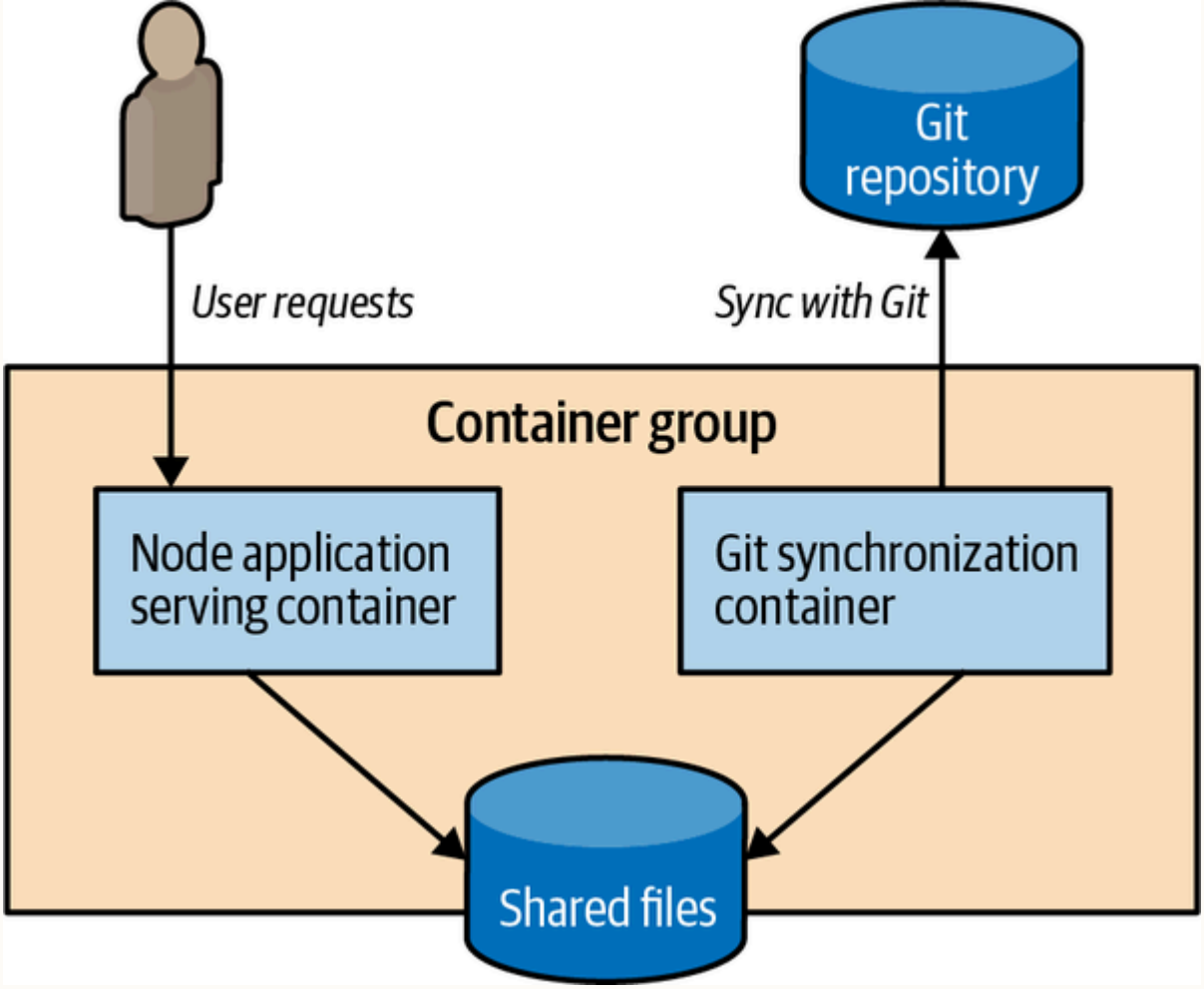


Figure 3-4. A simple sidecar-based PaaS

Designing Sidecars for Modularity and Reusability

In all of the examples of sidecars that we have detailed throughout this chapter, one of the most important themes is that every one was a modular, reusable artifact. To be successful, the sidecar should be reusable across a wide variety of applications and deployments. By achieving modular reuse, sidecars can dramatically speed up the building of your application.

However, this modularity and reusability, just like achieving modularity in high-quality software development, requires focus and discipline. In particular, you need to focus on developing three areas:

- Parameterizing your containers
- Creating the API surface of your container
- Documenting the operation of your container

Once you follow these common approaches, you will develop sidecars that can be reused across a wide variety of applications.

Parameterized Containers

Parameterizing your containers is the most important thing you can do to make your containers modular and reusable regardless of whether they are sidecars or not, though sidecars and other add-on containers are especially important to parameterize.

What do I mean when I say "parameterize"? Consider your container as a function in your program. How many parameters does it have? Each parameter represents an input that can customize a generic container to a specific situation. Consider, for example, the SSL add-on sidecar deployed previously. To be generally useful, it likely needs at least two parameters: the first is the name of the certificate being used to provide SSL, and the other is the port of the "legacy" application server running on localhost. Without these parameters, it is hard to imagine this sidecar container being usable for a broad array of applications. Similar parameters exist for all of the other sidecars described in this chapter.

Now that we know the parameters we want to expose, how do we actually expose them to users, and how do we consume them inside the container. There are two ways in which such parameters can be passed to your container: through environment variables or the command line. Though either is feasible, I have a general preference for passing parameters via environment variables. An example of passing such parameters to a side-car container is:

```
docker run -e=PORT=<port> -d <image>
```

Of course, delivering values into the container is only part of the battle. The other part is actually using these variables inside the container. Typically, to do that, a simple shell script is used that loads the environment variables supplied with the sidecar container and either adjusts the configuration files or parameterizes the underlying application.

For example, you might pass in the certificate path and port as environment variables:

```
docker run -e=PROXY_PORT=8080 -e=CERTIFICATE_PATH=/path/to/cert.crt ...
```

In your container, you would use those variables to configure an `nginx.conf` file that points the web server to the correct file and proxy location.

Define Each Container's API

Given that you are parameterizing your containers, it is obvious that your containers are defining a "function" that is called whenever the container is executed. This function is clearly a part of the API that is defined by your container, but there are other parts to this API as well, including calls that the container will make to other services as well as traditional HTTP or other APIs that the container provides.

As you think about defining modular, reusable containers, it is important to realize that all aspects of how your container interacts with its world are part of the API defined by that reusable container. As in the world of microservices, these *microcontainers* rely on APIs to ensure that there is a clean separation between the main application container and the sidecar. Additionally, the API exists to ensure that all consumers of the sidecar will continue to work correctly as subsequent versions are released. Likewise, having a clean API for a sidecar enables the sidecar developer to move more quickly since they have a clear definition (and hopefully unit tests) for the services they provide as a part of the sidecar.

To see a concrete example of why this API surface area is important, consider the configuration management sidecar we discussed earlier. A useful configuration for this sidecar might be `UPDATE_FREQUENCY`, which indicates how often the configuration should be synchronized with the filesystem. It is clear that if, at some later time, the name of the parameter is changed to `UPDATE_PERIOD`, this change would be a violation of the sidecar's API and clearly would break it for some users.

While that example is obvious, even more subtle changes can break the sidecar API. Imagine, for example, that `UPDATE_FREQUENCY` originally took a number in seconds. Over time and with feedback from users, the sidecar developer determined that specifying seconds for large time values (e.g., minutes) was annoying users and changed the parameter to accept strings (10 m, 5 s, etc.). Because old parameter values (e.g., 10, for 10 seconds) won't parse in this new scheme, this is a breaking API change. Suppose still that the developer anticipated this but made values without units parse to milliseconds where they had previously parsed to seconds. Even this change, despite not leading to an error, is a breaking API change for the sidecar since it will lead to significantly more frequent configuration checks and correspondingly more load on the cloud configuration server.

I hope this discussion has shown you that for true modularity, you need to be very conscious of the API that your sidecar provides, and that "breaking" changes to that API may not always be as obvious as changing the name of a parameter.

Documenting Your Containers

By now, you've seen how you can parameterize your sidecar containers to make them modular and reuseable. You've learned about the importance of maintaining a stable API to ensure that you don't break sidecars for your users. But there's one final step to building modular, reusable containers: ensuring that people can use them in the first place.

As with software libraries, the key to building something truly useful is explaining how to use it. There is little good in building a flexible, reliable modular container if no one can figure out how to use it. Sadly, there are few formal tools available for documenting container images, but there are some best practices you can use to accomplish this.

For every container image, the most obvious place to look for documentation is the `Dockerfile` from which the container was built. There are some parts of the `Dockerfile` that already document how the container works. One example of this is the `EXPOSE` directive, which indicates the ports that the image listens on. Even though `EXPOSE` is not necessary, it is a good practice to include it in your `Dockerfile` and also to add a comment that explains what exactly is listening on that port. For example:

```
...

# Main web server runs on port 8080
EXPOSE 8080
...
```

Additionally, if you use environment variables to parameterize your container, you can use the `ENV` directive to set default values for those parameters as well as document their usage:

```
...

# The PROXY_PORT parameter indicates the port on localhost to redirect
# traffic to.
ENV PROXY_PORT 8000
...
```

Finally, you should always use the `LABEL` directive to add metadata for your image; for example, the maintainer's email address, web page, and version of the image:

```
...

LABEL "org.opencontainers.image.vendor"="name@company.com"
LABEL "org.opencontainers.image.url"="http://images.company.com/my-cool-image"
LABEL "org.opencontainers.image.version"="1.0.3"
...
```

The names for these labels are drawn from the schema established by the Open Containers Initiative. The Open Containers Initiative (OCI) is the specification body which maintains the definition of what it means to be a container image. A part of that specification is the definition of a shared set of annotations/labels. By using a common taxonomy of image labels, multiple different tools can rely on the same meta information in order to visualize, monitor, and correctly use an application. By adopting shared terms, you can use the set of tools developed in the community without modifying your image. Of course, you can also add whatever additional labels make sense in the context of your image.

Summary

Over the course of this chapter, we've introduced the sidecar pattern for combining containers on a single machine. In the sidecar pattern, a sidecar container augments and extends an application container to add functionality. Sidecars can be used to update existing legacy applications when changing the application is too costly. Likewise, they can be used to create modular utility containers that standardize implementations of common functionality. These utility containers can be reused in a large number of applications, increasing consistency and reducing the cost of developing each application. Subsequent chapters introduce other single-node patterns that demonstrate other uses for modular, reusable containers.

Previous chapter

< II. Single-Node Patterns

Next chapter

4. Ambassadors >